Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 — Midterm Project

Author and team leader: Kyle Goodman

Team: Individual project

Repository: https://github.com/kgschool23/llm-authorship-classification

Abstract

This project investigates whether neural classifiers can identify which large language model generated a response. A balanced dataset was constructed from MixInstruct using responses from ChatGLM-6B, Flan-T5-XXL, and MPT-7B-Instruct. The dataset contains 18,000 responses to 6,000 shared prompts, with prompt groups kept separate across training, validation, and testing.

A convolutional neural network (CNN) and bidirectional long short-term memory network (LSTM) were evaluated using prompts alone, responses alone, and combined prompts and responses. Response-only test accuracy reached 96.04% for CNN and 95.93% for LSTM. Adding prompts improved LSTM accuracy to 97.67%, while CNN accuracy decreased slightly to 95.85%. Prompt-only accuracy was exactly 33.33%, consistent with the matched-prompt design.

A separate domain-transfer experiment excluded coding prompts from training and validation. Both models retained high accuracy on the held-out coding domain. Training-only fingerprint analysis also revealed substantial differences in response length, formatting, vocabulary, and lexical diversity. These findings demonstrate distinguishable patterns among the three selected checkpoints, although they do not establish universal fingerprints or explain causally which signals the classifiers use.

1. Problem Definition

The objective is three-class authorship classification: given text associated with an LLM interaction, predict whether its response came from ChatGLM, Flan-T5, or MPT. The task identifies the generating model rather than determining whether text is human-written.

The project investigates four research questions:

1. **RQ1**: Can CNN and LSTM classifiers identify an LLM from its response?
2. **RQ2**: Does including the user's prompt improve classification?
3. **RQ3**: Do learned patterns transfer to a task domain excluded from training?
4. **RQ4**: What linguistic or structural characteristics distinguish the selected models?

The experiments concern three specific checkpoints. Their results should not be interpreted as performance on every version of these families or on unseen LLMs.

2. Dataset Curation

2.1 Source and Construction

The data came from MixInstruct, released for the LLM-Blender project [1, 2]. MixInstruct contains prompts paired with candidate responses from multiple models. This project reused those existing responses rather than generating new responses through APIs.

The downloaded dataset was pinned to revision:

c7f77a4ef0515a99d1752c6387c22366d6922da6

The selected checkpoints were:

Classification label	**Upstream model identifier**
ChatGLM	chatglm-6b
Flan-T5	flan-t5-xxl
MPT	mpt-7b-instruct

Each record contains the required fields LLM_name, LLM_Input, and LLM_output. The prompt joins nonempty upstream instruction and input components with a newline. Model responses come from candidates[].text; the upstream top-level reference answer was not used as a model response.

The upstream validation and test files were pooled as a sampling source. New authorship splits were then constructed independently, so the upstream split names indicate provenance rather than this project's evaluation protocol.

2.2 Filtering and Responsible Data Handling

The source allowlist retained records originating from unified_chip2 and dolly_15k. Each retained prompt required exactly one nonempty response from every selected model. Complete prompt groups matching email-address or Social Security number patterns were excluded.

Prompts were deduplicated using Unicode normalization, case folding, and collapsed whitespace. This normalization determined grouping and identifiers; it did not replace the original classifier text.

The preparation process excluded:

Reason	**Prompt groups excluded**
Source outside the allowlist	2,402
Email or SSN pattern	21
Duplicate normalized prompt	207

The identifier-pattern screen is limited and should not be considered comprehensive anonymization. Raw and processed response text was excluded from Git tracking. The repository contains preparation code, provenance information, aggregate statistics, and predictions without raw response text.

2.3 Balanced, Prompt-Disjoint Splits

After filtering, 7,370 eligible prompt groups remained. A deterministic shuffle with seed 42 selected 6,000 groups. Every selected prompt contributed one response from each family.

Split	Prompt groups	Responses	Responses per family
Training	4,200	12,600	4,200
Validation	900	2,700	900
Test	900	2,700	900
Total	**6,000**	**18,000**	**6,000**

Splitting occurred at the prompt-group level. No normalized prompt appeared in more than one split. Consequently, the models could not encounter a training prompt paired with a different family's response during testing.

The selected prompts included 5,401 from unified_chip2 and 599 from dolly_15k. Some normalized response strings recurred across different prompts: 25 were shared between training and validation, 20 between training and test, and 11 between validation and test. These repeated strings were retained and documented.

2.4 Domain Labels

An ordered keyword heuristic assigned each prompt to coding, mathematics, writing, or other. These labels are approximate and were not manually validated. Across the selected dataset, the domain counts were:

Domain	Prompt groups
Coding	75
Mathematics	15
Writing	25
Other	5,885

"Other" is a catch-all category, not a verified factual-question-answering domain. Coding was selected for the domain-transfer experiment because it contained more prompt groups than the other named domains.

3. Preprocessing and Model Implementation

3.1 Text Representation

Only the designated text fields entered the classifiers. Model identifiers, source identifiers, domain labels, decoding metadata, and dataset scores were excluded from features.

The tokenizer lowercases text, separates Unicode word tokens and punctuation, and preserves line breaks through a newline token. Exact spacing and capitalization are not retained.

Each input mode uses a vocabulary fitted only on its training features, capped at 20,000 tokens including special tokens. Unknown validation and test tokens map to an unknown-token identifier.

The input modes were:

Mode	**Classifier input**
Input only	User prompt
Output only	Model response
Input + output	Prompt, separator token, response

Prompts were limited to 96 tokens and responses to 256 tokens. Combined inputs therefore contained at most 353 tokens, including the separator. Separate budgets ensured that a long prompt could not displace the entire response.

Batches used dynamic right padding while retaining each sequence's true length.

3.2 Convolutional Neural Network

The CNN uses a trainable 128-dimensional embedding followed by three parallel one-dimensional convolution branches. Kernel widths are 3, 4, and 5, with 128 filters per branch.

Each branch applies ReLU activation and masked global maximum pooling. The three pooled outputs are concatenated into a 384-dimensional feature vector, followed by dropout and a linear layer producing three class logits.

Masking prevents padded windows beyond the true sequence boundary from determining pooled features. Short sequences are supported using a single zero-padded window when necessary. Dropout probability is 0.3 at the embedding and final-feature stages.

This architecture can represent local token patterns at several widths.

3.3 Bidirectional LSTM

The required recurrent model is a one-layer bidirectional LSTM with a trainable 128-dimensional embedding and 128 hidden units in each direction.

The final forward and backward hidden states are concatenated into a 256-dimensional vector. Dropout with probability 0.3 precedes a linear three-class output layer.

Packed sequences exclude padded time steps from recurrent processing. Bidirectional processing is appropriate because the complete classification input is available when making a prediction.

Both models return raw logits for cross-entropy training. PyTorch provides the convolution and LSTM components used in these implementations [3, 4].

4. Training and Evaluation Protocol

All required configurations used the same fixed training settings:

Setting	Value
Random seed	42
Optimizer	Adam
Learning Rate	0.001
Batch size	32
Maximum epochs	20
Early-stopping patience	3 epochs
Checkpoint-selection metric	Validation macro-F1
Gradient clipping	Maximum norm of 1.0
Training loss	Cross-entropy
Numerical precision	Float32

The experiments ran on an NVIDIA GeForce RTX 4070 SUPER using PyTorch 2.9.1 with CUDA 12.8. Python, NumPy, and PyTorch random generators were seeded, and deterministic PyTorch operations were requested.

Each run saved its configuration, vocabulary, training history, selected checkpoint, dataset checksums, source checksums, validation metrics, and predictions. All six required models completed training before their selected checkpoints were evaluated on the test split. Test results were not used to retune hyperparameters.

Accuracy measures the proportion of correctly classified responses. Macro-F1 averages the three class-specific F1 scores equally.

Two baselines provide context:

- Uniform random prediction has expected accuracy of 33.33%.
- Predicting one constant class on the balanced test set gives 33.33% accuracy and macro-F1 of 0.1667.

The prompt-only experiment is also a matched-prompt control. Each identical prompt occurs once with every author label. A deterministic prompt-only prediction therefore correctly identifies exactly one of the three associated labels, producing 33.33% accuracy.

5. Results: RQ1 and RQ2

5.1 Six-Configuration Comparison

Model	Input	Selected epoch	Validation accuracy	Test accuracy	Test macro-F1
CNN	Prompt only	5	33.33%	33.33%	0.3273
CNN	Response only	14	97.26%	96.04%	0.9605
CNN	Prompt + response	2	96.52%	95.85%	0.9584
LSTM	Prompt only	2	33.33%	33.33%	0.3148
LSTM	Response only	6	97.15%	95.93%	0.9594
LSTM	Prompt + response	8	97.67%	97.67%	0.9767

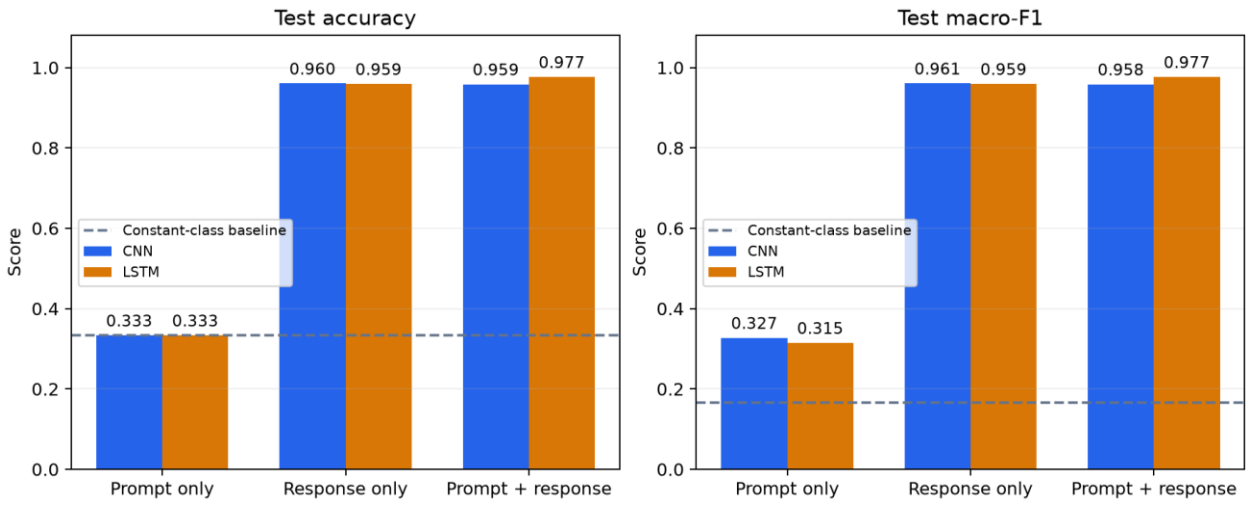


Figure 1. Test accuracy and macro-F1 for the six predefined architecture/input combinations.

5.2 RQ1: Identifying Authors From Responses

Both architectures identified the selected models accurately from responses alone. CNN correctly classified 2,593 of 2,700 responses, while LSTM correctly classified 2,590.

CNN's response-only accuracy exceeded LSTM's by approximately 0.11 percentage points, corresponding to three responses. This small difference does not establish that CNN is generally superior.

Response-only recall by family was:

Model	**ChatGLM recall**	**Flan-T5 recall**	**MPT recall**
CNN	95.89%	96.89%	95.33%
LSTM	95.56%	94.33%	97.89%

The results indicate recognizable response patterns across all three classes rather than success confined to one class.

5.3 RQ2: Whether Prompts Help

Prompt-only accuracy was exactly 33.33% for both architectures. This was expected from the matched-prompt construction, which removes prompt-distribution differences between author labels. Prompt-only macro-F1 differed between models because their predicted-class frequencies differed.

Adding the prompt had different effects on the architectures:

Model	Responses helped	Responses harmed	Net correct predictions	Accuracy change
CNN	48	53	-5	-0.19 percentage points
LSTM	75	28	+47	+1.74 percentage points

“Helped” means a response-only error became correct in the combined-input model. “Harmed” means a correct response-only prediction became incorrect.

The combined LSTM achieved the highest reported accuracy, 97.67%, with 63 errors. Its class recalls were 96.33% for ChatGLM and 98.33% for both Flan-T5 and MPT.

Prompt context may help the LSTM distinguish how different models respond to the same request. However, the experiment compares separately trained models, so the improvement cannot be attributed solely to a particular internal mechanism. Only one seed was tested.

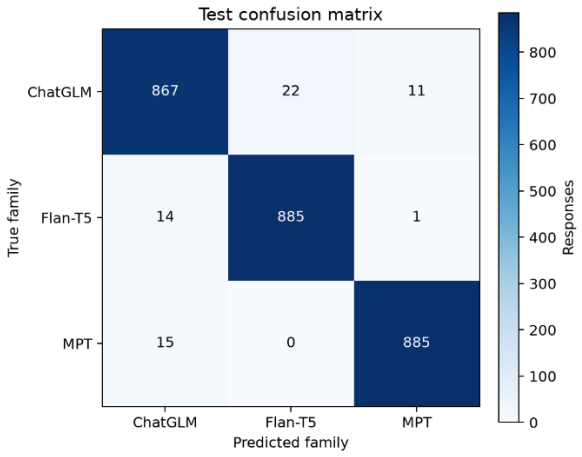


Figure 2. Test confusion matrix for the combined-input LSTM. Rows represent true authors and columns represent predicted authors.

5.4 Learning Behavior

The response-only CNN selected epoch 14 and stopped after 17 epochs. The response-only LSTM selected epoch 6 and stopped after 9 epochs. The combined LSTM selected epoch 8 and stopped after 11 epochs.

Response-only validation accuracy exceeded test accuracy by approximately 1.22 percentage points for both architectures. The combined LSTM's validation and test accuracies were both 97.67%. These results describe the observed split behavior and do not eliminate possible overfitting.

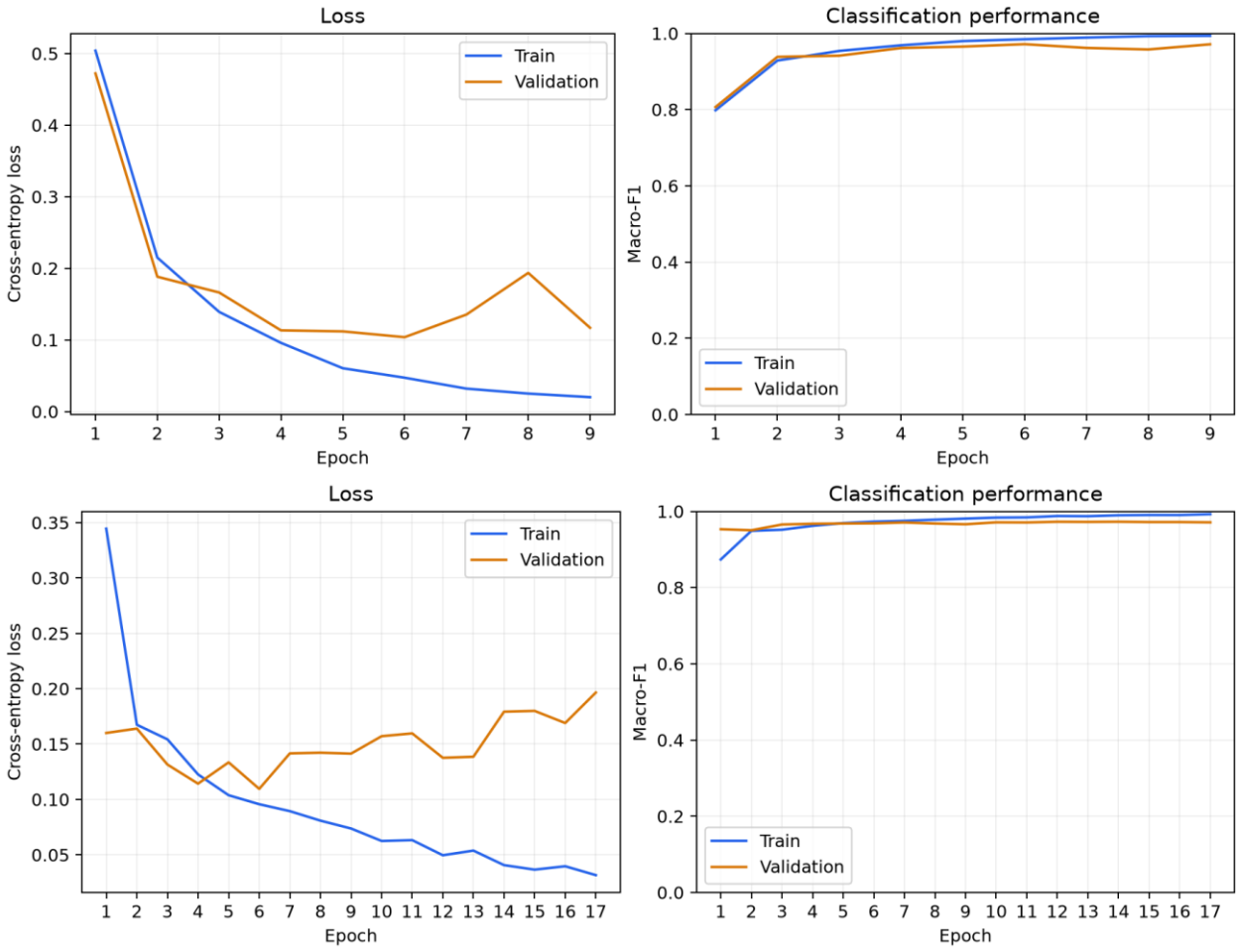


Figure 3. Training and validation learning curves for the response-only LSTM (top) and CNN (bottom). Checkpoints were selected using validation macro-F1.

6. RQ3: Generalization to Held-Out Coding Tasks

6.1 Experimental Design

Two additional response-only models were trained with all heuristically labeled coding prompts excluded from training and validation. These models used fresh weights and vocabularies fitted only to their new training data.

Coding prompts from the original training, validation, and test splits were pooled into a held-domain test set. Original noncoding test prompts formed the control set.

New split	Prompt groups	Responses
Noncoding training	4,144	12,432
Noncoding validation	893	2,679
Noncoding control test	888	2,664
Held-out coding test	75	225

Some coding test prompts had been training examples in the earlier RQ1/RQ2 experiment. They were excluded from the new RQ3 models' training and validation data, which is the relevant boundary for this experiment. The original six models and their results were unchanged.

Both additional models completed training before either evaluation set was scored. Hyperparameters remained fixed.

6.2 Domain-Transfer Results

Model	Evaluation set	Accuracy	Macro-F1
CNN	Noncoding control	96.40%	0.9638
CNN	Held-out coding	96.89%	0.9689
LSTM	Noncoding control	95.87%	0.9590
LSTM	Held-out coding	96.00%	0.9602

CNN made seven errors on the 225 coding responses; LSTM made nine. Coding accuracy was 0.49 percentage points higher than control accuracy for CNN and 0.13 points higher for LSTM.

Neither architecture showed an observed accuracy decrease on the held-out coding domain. This supports transfer of useful authorship patterns within the collected dataset.

However, the held-domain set contains only 75 independent prompt groups. Domain labels are heuristic, the control contains mixed noncoding tasks, and source composition or response-length differences may influence performance. The small increases should not be interpreted as statistically significant improvements or proof of universal domain invariance.

7. RQ4: Characteristics That Distinguish Models

Fingerprint analysis used only the original 12,600 training responses. It examined response length, line breaks, punctuation, lexical diversity, and vocabulary associations.

7.1 Length, Formatting, and Lexical Diversity

Training-response statistic	ChatGLM	Flan-T5	MPT
Median characters	856.5	36.0	1,113.0
Median word tokens	150	7	208
Median newlines per 100 word tokens	3.55	0.00	2.39
Median punctuation marks per 100 word tokens	16.36	6.45	15.05
Median unique-word percentage	56.29%	100.00%	50.79%

Word tokens are Unicode word-pattern matches, not linguistic word counts. Unique-word percentage is the number of distinct lowercased word tokens divided by the total word-token count.

Flan-T5 responses were substantially shorter and contained no newline characters in the analyzed training set. ChatGLM and MPT produced longer responses with more line structure and punctuation relative to their token counts.

Flan-T5's higher lexical-diversity ratio should not be interpreted as richer vocabulary. Short responses have fewer opportunities to repeat words, making a 100% unique-word ratio easier to obtain.

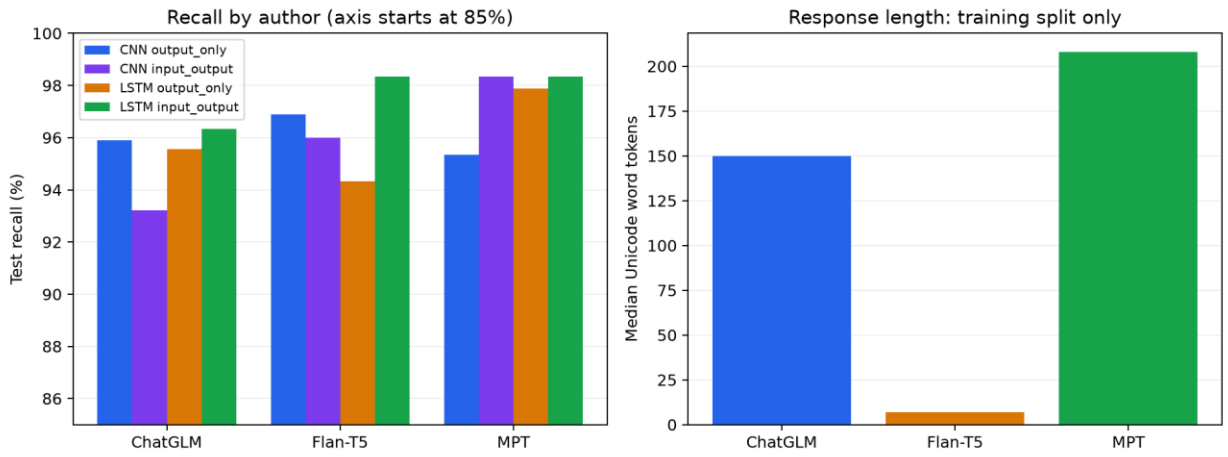


Figure 4. Per-author test recall for response-based classifiers and median training-response length. The recall panel uses an axis beginning at 85%.

7.2 Vocabulary Associations

Tokens were ranked by their additive-one-smoothed log frequency in one family compared with the other two. Candidate tokens required at least 25 occurrences across at least 10 responses from that family.

Examples of relatively associated tokens included:

Family	**Example associated tokens**
ChatGLM	ultimately, noting, additionally
Flan-T5	yes, click, inside
MPT	printf, char, stdio

These associations describe the collected responses. They may reflect verbosity, preferred expressions, generated code, task handling, or generation artifacts. They are not permanent signatures of the model families.

7.3 Interpretation

The dataset contains both structural and lexical differences that could support classification. In particular, Flan-T5's short responses offer a strong potential structural cue. Preserved punctuation and newline tokens also make formatting differences available to the classifiers.

Nevertheless, descriptive fingerprints do not establish which signals the trained networks actually use. No signal-removal ablation or causal attribution experiment was performed. A useful extension would remove formatting or control response length, retrain the models, and measure how performance changes.

8. Limitations

The main limitations are:

- **Checkpoint scope**: Results concern three specific checkpoints, not all models or versions within their families.
- **Single seed**: Differences between configurations are descriptive; repeated-seed uncertainty was not measured.
- **Dependent observations**: Three responses share each prompt. Response rows are therefore not independent observations.
- **Dataset artifacts**: Length, decoding settings, tuning, language, repeated phrases, and source composition may contribute to classification.
- **Response overlap**: Some normalized response strings appear across splits despite prompt disjointness.
- **Truncation**: Only the first 256 response tokens enter response-based models.
- **Domain uncertainty**: Domain labels are heuristic, and the coding transfer set is small.
- **Interpretability**: Fingerprint statistics describe the data rather than prove causal model behavior.

Future experiments could include additional model families, manually verified domains, repeated seeds, response-length controls, and formatting-removal ablations.

9. Lessons and Experience

A major lesson was that dataset construction determines how an experiment should be interpreted. Matching prompts across authors made prompt-only classification a useful leakage control. Its 33.33% accuracy was a consequence of the experimental design rather than evidence of failed training.

Preventing leakage required more than separating CSV files. Prompt groups had to remain intact across splits, vocabularies had to be fitted only on training text, and metadata had to be excluded from classifier inputs.

Padding also required explicit treatment. The CNN needed masked pooling, while the LSTM needed packed sequences. Verification covered short inputs, finite gradients, padding invariance, vocabulary serialization, and batch ordering.

Reproducibility checks proved useful during recovery from a Windows path-format error in evaluation. Saved checkpoints allowed testing to continue without retraining, while checksums verified that the training sources and datasets were preserved.

Finally, high accuracy alone did not explain model behavior. The fingerprint analysis exposed large structural differences between authors, while the domain experiment tested whether useful patterns persisted beyond the training task mixture. Combining these analyses produced a more informative result than reporting accuracy alone.

The project includes 30 unit tests covering preprocessing, dataset integrity, model behavior, analysis, and domain exclusion, together with CUDA computation and model smoke checks.

10. Conclusion

CNN and LSTM classifiers accurately identified the three selected LLM checkpoints from their responses, achieving approximately 96% response-only test accuracy.

Prompts alone provided no authorship information under the matched-prompt design. Adding prompts improved the LSTM to 97.67% accuracy but slightly reduced CNN accuracy, showing that prompt context did not help both architectures equally.

Separate models trained without coding prompts retained high accuracy on held-out coding responses. Training-only analysis also identified pronounced differences in length, formatting, lexical diversity, and vocabulary.

All four research questions were investigated. The results support distinguishable and transferable patterns within this dataset, while the limited model selection, single seed, heuristic domains, and potential structural shortcuts constrain broader conclusions.

References

[1] Jiang, D., Ren, X., and Lin, B. Y. (2023). LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 14165–14178. https://aclanthology.org/2023.acl-long.792/

[2] LLM-Blender. MixInstruct dataset and dataset card. https://huggingface.co/datasets/llm-blender/mix-instruct

Dataset revision used: c7f77a4ef0515a99d1752c6387c22366d6922da6.

[3] PyTorch. Conv1d documentation, version 2.9. https://docs.pytorch.org/docs/2.9/generated/torch.nn.Conv1d.html

[4] PyTorch. LSTM documentation, version 2.9. https://docs.pytorch.org/docs/2.9/generated/torch.nn.LSTM.html

AI Assistance Disclosure

ChatGPT assisted with project planning, code scaffolding, debugging, experiment design, interpretation, and report drafting. Experiments were executed locally, and saved predictions were checked against reported metrics. The submitted implementation and conclusions remain the author's responsibility.